

Cấu hình và Giao tiếp Ngoại vi GPIO trên STM32F103

Đặng Đức Huy*

Khoa Điện - Điện tử
Đại học Bách Khoa TP.HCM (HCMUT)

*Email: duchuydangbd@gmail.com

Machine Learning & IoT Lab (ML IoT)
Ho Chi Minh City University of Technology (HCMUT)

1. Phần I: Khởi động
2. Phần II: Kiến trúc phần cứng & GPIO
3. Phần III: Thực hành Digital Output
4. Phần IV: Thực hành Digital Input
5. Phần V: Ma trận bàn phím 4x4
6. Phần VI: Homework



Thông số	Chi tiết
CPU	ARM Cortex-M3, 72 MHz
Flash	64KB
SRAM	20KB
ADC	2 × 12-bit, 16 kênh
Timers	7 (bao gồm PWM, watchdog)
Giao tiếp	3 USART, 2 SPI, 2 I ² C, CAN, USB
GPIO	37 chân, 5 V tolerant
Điện áp	2.0 – 3.6 V
Debug	SWD, JTAG

So sánh STM32F103C8T6 và ATmega324P

MCU	STM32F103C8T6 (ARM Cortex-M3)	ATmega324P(AVR)
Độ rộng bus dữ liệu	32-bit	8-bit
Xung nhịp tối đa	72 MHz	20 MHz
Cơ chế cấp xung ngoại vi	Clock Gating	Static Clock
Mức điện áp hoạt động	3.3V	5V
Bộ nhớ (Flash / SRAM)	64KB / 20KB	32KB / 2KB

Hiệu năng và Năng lượng

- ▶ Thực trạng trên các dòng 8-bit (như AVR): Xung nhịp được cấp tính và liên tục cho toàn bộ các khối ngoại vi ngay từ khi cấp nguồn, bất kể chúng có đang được sử dụng hay không.
- ▶ Vấn đề khi mang lên ARM 32-bit:
 - ◀ Tốc độ xung nhịp cực cao (lên đến 72MHz).
 - ◀ Số lượng ngoại vi tích hợp bên trong chip rất lớn.
- ▶ Hậu quả: Nếu vẫn duy trì cơ chế cấp xung liên tục như AVR, vi điều khiển sẽ tiêu tốn một lượng điện năng khổng lồ một cách vô ích, sinh nhiệt cao và hoàn toàn không phù hợp cho các thiết bị nhúng/IoT chạy pin.
- ▶ Câu hỏi đặt ra: Làm thế nào để giữ được sức mạnh xử lý 72MHz nhưng vẫn tối ưu hóa được năng lượng tiêu thụ?

Khắc phục bằng cơ chế Clock Gating

- ▶ Giải pháp: Khối Reset and Clock Control (RCC) được sinh ra để làm "người gác cổng", đóng vai trò quản lý độc lập việc phân phối xung nhịp.
- ▶ Nguyên lý hoạt động:
 - ◀ Mặc định TẮT: Mọi ngoại vi đều bị ngắt xung nhịp (đưa vào trạng thái "ngủ") để tiết kiệm điện tối đa. Chủ động BẬT: Người lập trình bắt buộc phải "mở khóa" xung nhịp cho đúng ngoại vi nào thực sự cần dùng.
- ▶ Trade-off: Bạn sẽ tốn thêm một dòng code (Clock Activation), nhưng hệ thống đạt được sự cân bằng hoàn hảo giữa hiệu năng xử lý mạnh mẽ và khả năng tiết kiệm năng lượng.

- ▶ Cấu trúc liên kết: Khối ngoại vi GPIO (bao gồm các Port A, B, C...) được treo trực tiếp trên đường bus ngoại vi tốc độ cao APB2.
- ▶ Nguyên tắc lập trình: Quy trình bắt buộc đầu tiên trong mọi mã nguồn cấu hình phần cứng là phải kích hoạt xung nhịp cho bus APB2 tại Port tương ứng.

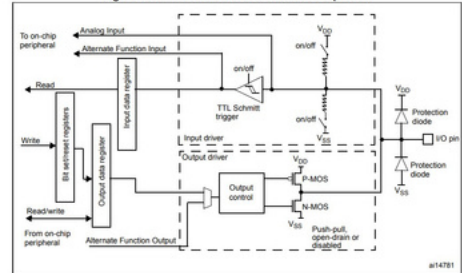
Khi làm việc với bất kỳ ngoại vi nào trên MCU lõi ARM (GPIO, UART, SPI, I2C...), ta luôn tuân theo 3 bước cơ bản:

1. Clock Activation: Bật nguồn xung nhịp cho ngoại vi thông qua hệ thống bus (APB/AHB). Nếu không cấp clock, ngoại vi sẽ không hoạt động.
2. Hardware Configuration: Chọn chức năng cho chân (Input, Output, Analog, hoặc Alternate Function) và cấu hình chân (Push-Pull, Open-Drain, Pull-up, Pull-down).
3. Application Logic: Viết thuật toán điều khiển hoặc lấy mẫu dữ liệu, thường chạy trong vòng lặp vô hạn while(1) để xử lý liên tục theo thời gian thực.

Công tắc điện tử

- ▶ MOSFET hoạt động như một chiếc công tắc, nhưng không dùng lực cơ học mà dùng điện áp để điều khiển việc đóng/cắt dòng điện.
- ▶ PMOS: Dẫn điện khi chân điều khiển (Gate) ở mức logic 0 (0V). Ngắt khi ở mức 1. Thường dùng để "đẩy" nguồn VCC.
- ▶ NMOS: Dẫn điện khi chân điều khiển (Gate) ở mức logic 1 (3.3V). Ngắt khi ở mức 0. Thường dùng để "kéo" xuống GND.

Figure 13. Basic structure of a standard I/O port bit



Chế độ Push-Pull

- ▶ Điều khiển bởi hai Transistor PMOS (trên) và NMOS (dưới) hoạt động đối nghịch.
- ▶ Xuất mức 1 (3.3V): PMOS dẫn, NMOS ngắt → dòng điện được "đẩy" ra.
- ▶ Xuất mức 0 (0V): PMOS ngắt, NMOS dẫn → dòng điện được "kéo" xuống GND.
- ▶ Ứng dụng: Điều khiển LED, truyền tín hiệu tốc độ cao.

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register		
General purpose output	Push-pull	0	0	01	10	0 or 1		
	Open-drain		1			0 or 1		
Alternate Function output	Push-pull	1	0	11	see Table 21	Don't care		
	Open-drain		1			Don't care		
Input	Analog	0	0	00		Don't care		
	Input floating		1			Don't care		
	Input pull-down	1	0			0		
	Input pull-up					1		

Chế độ Open-Drain

- ▶ Transistor PMOS phía trên bị loại bỏ hoàn toàn khỏi cấu trúc điều khiển.
- ▶ Xuất mức 0: NMOS dẫn → chân nối xuống GND.
- ▶ Xuất mức 1: NMOS ngắt → trạng thái trở kháng cao (Hi-Z), lơ lửng điện áp.
- ▶ Bắt buộc nối thêm trở kéo lên bên ngoài để đạt mức 1.
- ▶ Ứng dụng: Giao tiếp bus I2C, chuyển đổi mức điện áp.

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register		
General purpose output	Push-pull	0	0	01 10 11 see Table 21		0 or 1		
	Open-drain		1			0 or 1		
Alternate Function output	Push-pull	1	0					Don't care
	Open-drain		1					Don't care
Input	Analog	0	0	00				Don't care
	Input floating		1					Don't care
	Input pull-down	1	0			0		
	Input pull-up					1		

Input Floating

- ▶ Xảy ra nếu cấu hình chân là Input nhưng không có trở kéo cố định.
- ▶ Kết quả: Giá trị đọc về tại thanh ghi dữ liệu vào (IDR) sẽ nhảy hỗn loạn giữa 0 và 1, không kiểm soát được.

Cách cấu hình

- ▶ Để kích hoạt điện trở nội, thiết lập bits $CNF[1:0] = 10$ (Input with pull-up / pull-down).
- ▶ Thanh ghi PxODR quyết định loại điện trở được sử dụng.
- ▶ Thiết lập PxODR = 1: Chọn chế độ Pull-up.
- ▶ Thiết lập PxODR = 0: Chọn chế độ Pull-down.

Đề bài

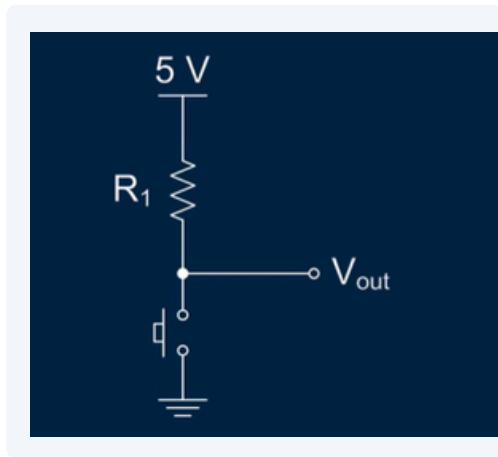
Cấu hình chân PC13 làm ngõ ra và viết chương trình nháy LED (LED sáng 500ms, tắt 500ms, lặp lại liên tục).

Bản chất vật lý

- ▶ Nút bấm là một cấu trúc cơ khí đàn hồi.
- ▶ Khi có lực tác động, các tiếp điểm kim loại không đóng khít ngay lập tức mà bị nảy liên tục trong khoảng 5ms . . . 20ms.
- ▶ Vi điều khiển thực thi lệnh ở tốc độ cực cao, chuỗi xung nhiễu này sẽ bị nhận nhầm thành hàng chục hành vi bấm nút liên tục.

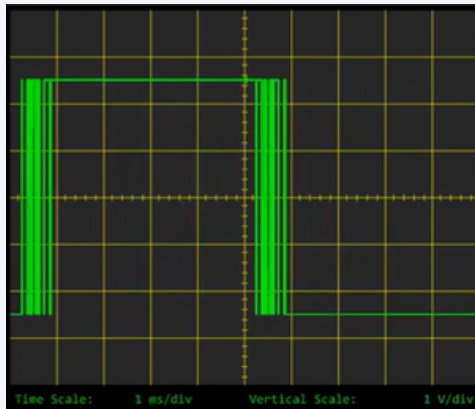
Hiện tượng dội phím cơ học

- ▶ Nhìn vào sơ đồ mạch: Đây là mạch cấu hình nút bấm cơ bản với điện trở kéo lên bên ngoài (R_1). Mặc định khi chưa bấm, V_{out} ở mức cao (5V). Khi bấm nút, V_{out} đập xuống mức thấp (0V).



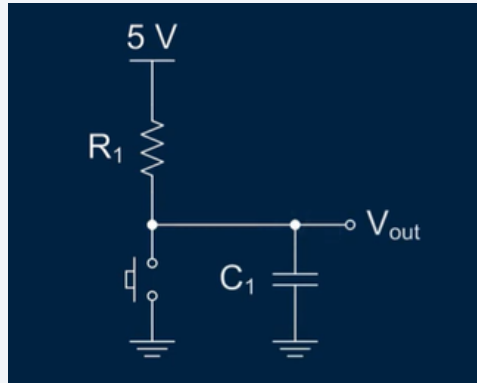
Hiện tượng dội phím cơ học

- ▶ Nhìn vào dạng sóng trên Oscilloscope: Ở hai sườn chuyển đổi trạng thái (từ thấp lên cao và từ cao xuống thấp), các bạn sẽ thấy điện áp không đi theo một đường thẳng dứt khoát. Thay vào đó, nó tạo thành một "rừng xung" dày đặc dao động liên tục trong khoảng 1ms đến 2ms.
- ▶ Hậu quả: Bộ vi điều khiển xử lý rất nhanh, nó sẽ đọc chuỗi xung nhiễu này và hiểu nhầm rằng người dùng đang bấm nút liên tục hàng chục lần.



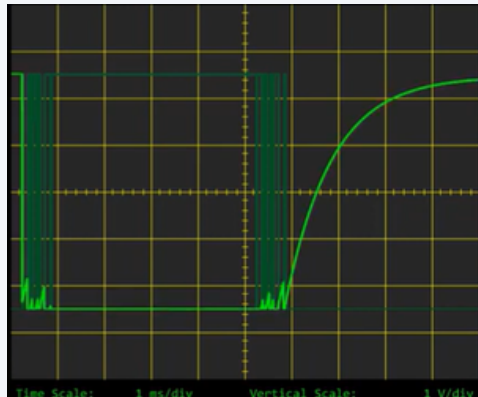
Giải thuật

- ▶ Nhìn vào sơ đồ mạch cải tiến: Chúng ta mắc thêm một tụ điện C_1 ($0.1\mu F$) song song với nút bấm (nối từ V_{out} xuống Đất).



Giải thuật

- ▶ Nhìn vào dạng sóng trên Oscilloscope: Tại sườn xuống (khi bấm nút): Xung nhiễu nảy phím đã được tụ điện hấp thụ đáng kể, ép biên độ dao động nhỏ lại sát đường 0V, giúp tín hiệu "sạch" hơn rất nhiều.
- ▶ Tại sườn lên (khi nhả nút): Dạng sóng vuông sắc cạnh ban đầu đã biến thành một đường cong sọc điện dung (đường cong RC) mượt mà. Tụ điện C1 sẽ được nạp điện từ từ thông qua điện trở R1, kéo dài thời gian điện áp tăng từ 0V lên 5V. Nhờ đường cong này, toàn bộ các gai nhiễu tần số cao do tiếp điểm cơ khí nảy ra khi nhả nút đã bị san phẳng hoàn toàn. Ngoài ra, ta kết hợp Delay (khoảng 20-30 ms) để vi điều khiển đọc được dữ liệu sạch nhất.



Đề bài

Cấu hình chân PA0 làm ngõ vào (Input Pull-up) để đọc trạng thái một nút nhấn, có áp dụng giải thuật chống dội vừa học.

Vì sao phải dùng ma trận?

- ▶ Nếu kết nối mỗi nút với 1 chân GPIO độc lập, một bàn phím 16 phím sẽ tiêu tốn 16 chân — cạn kiệt tài nguyên chân của vi điều khiển.
- ▶ Sắp xếp 16 nút bấm thành lưới 4 Hàng (Rows) $\times$ 4 Cột (Cols), mỗi nút nằm tại giao điểm của một Hàng và một Cột.
- ▶ Số chân tiêu tốn chỉ còn $4 + 4 = 8$ chân — tiết kiệm được $16 - 8 = 8$ chân so với cách đấu độc lập.

	Cot1(PA4)	Cot2(PA5)	Cot3(PA6)	Cot4(PA7)
Hang1(PA0	-----+---[K1]-----+---[K2]-----+---[K3]-----+---[K4]			
)	-----+---[K5]-----+---[K6]-----+---[K7]-----+---[K8]			
Hang2(PA1	- - - - - + - - - [K9] - - - - + - - [K10] - - - + - - [K11] - - - + - - [K12]			
)	- - - - - + - - - [K13] - - - + - - [K14] - - - + - - [K15] - - - + - - [K16]			

Hang3(PA

2) Tọa độ mỗi phím được xác định duy nhất bởi cặp [Hàng][Cột], ví dụ K7 nằm tại Hàng 2, Cột 3.

Hang4(PA

3)

- ▶ 4 chân Hàng (PA0-PA3) cấu hình làm GPIO_Output Push-Pull.
- ▶ 4 chân Cột (PA4-PA7) cấu hình làm GPIO_Input Pull-up.

- ▶ Cơ chế quét: CPU lần lượt kéo duy nhất một Hàng xuống mức logic 0 tại một thời điểm, 3 Hàng còn lại giữ ở mức 1.
- ▶ Lấy mẫu: CPU đọc đồng thời trạng thái của cả 4 Cột ứng với Hàng đang được kéo xuống 0.
- ▶ Xác định: Nếu phím tại giao điểm của Hàng đang quét và Cột đang đọc được nhấn, mức 0 từ Hàng sẽ kéo điện áp Cột sụt xuống 0.
- ▶ Chu kỳ đầy đủ: Lặp lại quy trình trên tuần tự cho cả 4 Hàng (Hàng 1 → Hàng 2 → Hàng 3 → Hàng 4) để quét hết toàn bộ 16 phím.

```
uint16_t ROW_PINS[4] = {GPIO_PIN_0, GPIO_PIN_1, GPIO_PIN_2, GPIO_PIN_3}; uint16_t  
COL_PINS[4] = {GPIO_PIN_4, GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7};  
  
uint8_t KEYPAD_4x4_Scan(void)  
{  
    for (uint8_t r = 0; r < 4; r++) {  
        for (uint8_t i = 0; i < 4; i++) // Chi dap Hang r xuong 0  
            HAL_GPIO_WritePin(GPIOA, ROW_PINS[ i ],  
                               (i == r) ? GPIO_PIN_RESET : GPIO_PIN_SET);  
        HAL_Delay ( 1 );  
  
        for (uint8_t c = 0; c < 4; c++) {  
            if (HAL_GPIO_ReadPin(GPIOA, COL_PINS[c]) == GPIO_PIN_RESET) {  
                HAL_Delay(20); // chong doi  
                if (HAL_GPIO_ReadPin(GPIOA, COL_PINS[c]) == GPIO_PIN_RESET)  
                    return (r * 4 + c) + 1; // ma phim 1..16  
            }  
        }  
    }  
    return 0; // khong co phim nao duoc nhan  
}
```

Đề bài

Cấu hình PA0-PA3 làm Hàng, PA4-PA7 làm Cột cho ma trận phím 4x4 (16 phím). Sử dụng lại hàm `KEYPAD_4x4_Scan()` đã phân tích.

- ▶ Khi một phím được nhấn hợp lệ, hiển thị mã số phím (1-16) tương ứng ra LED hoặc UART.

Thank You